



Testudo

User Manual

A native, local-first macOS environment for structured professional and research work

Manual version: 1.6
Application version: 0.1.10 (build 11)
Platform: macOS 26+, Apple Silicon
Date: October 2026

About this manual

This manual documents the implemented behaviour of **Testudo** 0.1.10 (build 11). It is intentionally more than a feature list. It is written so that a new user can understand the data model, perform ordinary work, and actively verify that the application behaves as expected.

Each major chapter therefore contains practical procedures and **Try it** exercises. These exercises are safe to perform in the Demo Environment or in a disposable Work Environment. They are especially useful when learning the application after an update.

Version covered

This edition is aligned with Testudo 0.1.10. The release expands full-pane creation and editing for People, Themes, Organizations and Groups; extends Parent Task relationship inheritance to child Notes and Activities as well as Sub-tasks; adds explicit historical Completed date, time and time-zone editing for Tasks; and introduces native Markdown rendering for Work descriptions and structural Notes. Existing `.testudoenv` data remains compatible because these changes do not require a new storage representation.

Testudo is an early public release. Keep independent backups of important `.testudoenv` packages. When experimenting with unfamiliar operations, prefer the Demo Environment first.

Terminology

An **Activity** is a chronological Work item that records something that happened. A **Calendar Event** is a separate scheduled object. The two concepts are deliberately different even when they refer to the same real-world work.

Contents

About this manual	i
1 What Testudo Is	1
1.1 Purpose	1
1.2 The main domains	1
1.3 Local-first architecture	1
2 Requirements, Installation and Updating	3
2.1 Supported baseline	3
2.2 Quick installation	3
2.3 Compatibility check only	3
2.4 Manual source build	3
2.5 Updating	4
3 First Launch and the Local Testudo Profile	5
3.1 Create or restore a profile	5
3.2 Importing a portable profile	5
3.3 Application password and recovery	5
4 Local Log Out, Local Sign Out and Environment Sign Out	6
4.1 Log Out of Testudo	6
4.2 Sign Out from Testudo	6
4.3 Sign Out of Environment	6
5 Work Environments	7
5.1 The Work Environments screen	7
5.2 Creating an Environment	7
5.3 Opening an existing Environment	7
5.4 Environment login	7
5.5 Forgetting an Environment from one Mac	8
6 Environment Membership and Administration	9
6.1 Environment roles	9
6.2 Membership identity	9
6.3 Environment passwords	9
6.4 Environment recovery questions	9
7 The Three-Column Interface	10
7.1 Sidebar	10
7.2 Middle column	10
7.3 Detail pane	10
7.4 Sidebar visibility	10
7.5 Back and Forward navigation	10
8 Work: Tasks, Notes and Activities	12
8.1 Three Work types	12
8.2 Creating Work	12
8.3 Editing existing Work	12
8.4 Creating child Work quickly	12
9 Task Workflow Status	14
9.1 The four statuses	14
9.2 To Do	14

9.3 In Progress	14
9.4 Completed	14
9.5 Changing status from the detail view	14
10 Discontinued Tasks	15
10.1 Why Discontinued exists	15
10.2 Required metadata	15
10.3 What remains visible	15
10.4 Historical cycles	15
11 Closing, Archive and Reopening	16
11.1 Closed is not a status	16
11.2 What closing does	16
11.3 Archive presentation	16
11.4 Reopen	16
11.5 Backward compatibility	16
12 Notes	18
13 Activities and the Work Log	19
13.1 Activity semantics	19
13.2 Activity in Timeline and History	19
14 Themes and Multi-Theme Classification	20
14.1 Purpose	20
14.2 Theme detail fields	20
14.3 Multiple Themes on one Work item	20
14.4 Descendant aggregation	20
15 People	21
15.1 Person profile fields	21
15.2 Affiliations	21
15.3 Environment membership link	21
16 Organizations and Groups	22
16.1 Organizations	22
16.2 Groups	22
16.3 Affiliated People	22
17 Work Relationships	23
17.1 Inheritance by child items	23
18 Related Work	24
19 Today	25
19.1 Today is a work summary, not only a schedule	25
19.2 Needs Attention	25
19.3 Upcoming	25
20 Task Lists and Timeline	26
20.1 Timeline	26
21 History and the Work Log	27
21.1 Audit timestamps	27
21.2 Log events	27
21.3 Deletion and history	27
22 Calendar Overview	28
22.1 What appears in Calendar	28
22.2 Calendar sources	28

23 Creating and Editing Calendar Events	29
23.1 New Calendar Event	29
23.2 Calendar Event status	29
23.3 External events	29
24 Apple Calendar Integration	30
24.1 Connecting Apple Calendar	30
24.2 Permission and availability	30
25 iCal Subscriptions	31
26 Time Zones and Historical Dates	32
26.1 Absolute time and source zone	32
26.2 Historical entry	32
27 Demo Environment	33
27.1 Purpose	33
27.2 Demo naming	33
27.3 Suggested tour	33
28 Storage, Portability and Backups	34
28.1 Portable Environment package	34
28.2 Cloud-synchronised folders	34
28.3 Backups	34
29 Security and Privacy	35
29.1 Passwords	35
29.2 Security questions	35
29.3 External calendar secrets	35
29.4 Sensitive portable files	35
30 Deletion Semantics	36
31 Recommended Working Practices	37
31.1 Separate plan, context and evidence	37
31.2 Use Discontinued instead of pretending success	37
31.3 Close only when the top-level work is no longer part of active workflow	37
31.4 Use Themes for concepts, not as a duplicate Task tree	37
31.5 Model recurring professional context explicitly	37
31.6 Record outcomes as Activities	37
32 Troubleshooting	38
32.1 The installer says the source checkout has local changes	38
32.2 Apple developer tools are missing	38
32.3 An Environment package was moved	38
32.4 Environment login fails	38
32.5 Apple Calendar does not appear	38
32.6 iCal synchronisation fails	39
32.7 A Task cannot be Closed	39
32.8 A Discontinued status change appears unavailable in a shortcut	39
32.9 Archived Work is missing from active lists	39
33 Guided End-to-End Verification Lab	40
33.1 Step 1: create the Environment	40
33.2 Step 2: create professional context	40
33.3 Step 3: create a top-level Task	40
33.4 Step 4: create children	40
33.5 Step 5: exercise status and Today	41
33.6 Step 6: exercise Discontinued	41
33.7 Step 7: exercise Completed and Archive	41

33.8 Step 8: navigation history	41
33.9 Step 9: Reopen	41
33.10 Step 10: calendar	41
34 Technical Reference	42
34.1 Important file extensions	42
34.2 Source tree	42
34.3 Build scripts	42
34.4 Version identity	42
34.5 Task status persistence compatibility	43
35 Glossary	44
A Command Reference	45
A.1 Check compatibility	45
A.2 Install	45
A.3 Clone manually	45
A.4 Update source	45
A.5 Build local installer	45
A.6 Build application only	45
A.7 Request Apple Command Line Tools	45
B Manual Source and Compilation	46
C Quick Acceptance Checklist	47
D Early Release Notice	48
License	49
Author	50

What Testudo Is

1.1 Purpose

Testudo is a native macOS application for work that is difficult to represent with a flat task list. It combines actionable work, durable context, a chronological work log, professional entities, semantic classification, calendars, and local portable storage.

Typical use cases include:

- scientific research and laboratory work;
- research software engineering;
- technical and infrastructure operations;
- academic collaborations and committees;
- long-running projects with many sub-problems;
- work involving multiple institutions, teams or people;
- activity where preserving the history of decisions and outcomes is important.

The core design principle is that **structure** and **chronology** are both first-class. A Task can have children and Themes, but Testudo also preserves when the work started, completed, was discontinued, was closed, or was reopened.

1.2 The main domains

Testudo connects several domains that are often kept in separate applications:

Domain	Purpose
Work	Tasks, Notes and Activities, including unlimited Task nesting.
Themes	Hierarchical semantic classification, independent from Task hierarchy.
Entities	People, Groups and Organizations describing professional context.
Relationships	Semantic links such as For, Requested by, With, Assigned to and Related to.
Calendar	Testudo events plus read-only external calendar sources.
History	Significant lifecycle and relationship events.
Work Environments	Portable, independent <code>.testudoenv</code> packages.
Identity	A local Testudo profile plus separate Environment memberships.

1.3 Local-first architecture

Testudo does not require a Testudo cloud service. Work data lives in the selected `.testudoenv` package. The application also stores installation-level information such as the local user profile, registered Environment locations, filesystem bookmarks and remembered sign-ins.

This separation matters:

- copying a `.testudoenv` package copies the Work Environment;

- signing out from the local Testudo installation does not delete the Environment package;
- forgetting an Environment from one Mac does not delete its package;
- a cloud-synchronised folder can transport the package, but Testudo is still operating on files rather than a central multi-user server.

After opening Testudo, identify: (1) your local Testudo user, (2) the active Work Environment, and (3) your Environment membership. These are related but separate identities.

Requirements, Installation and Updating

2.1 Supported baseline

The current source targets:

- macOS 26 or later;
- Apple Silicon;
- an Apple developer toolchain with Swift 6 or later;
- Git.

The repository uses a source-first installation model. The application is compiled on the Mac where it is used and is ad-hoc signed by the local build script.

2.2 Quick installation

Run:

```
curl -fsSL \  
  https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh \  
  | bash
```

The installer checks the machine, clones or updates the repository, refuses to overwrite a checkout with local modifications, builds Testudo, creates a DMG, verifies it and opens the finished installer.

When the DMG opens, drag **Testudo.app** into **/Applications**.

2.3 Compatibility check only

To test the toolchain without cloning or building:

```
curl -fsSL \  
  https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh \  
  | bash -s -- --check
```

2.4 Manual source build

```
git clone https://github.com/foivoskar/Testudo.git  
cd Testudo  
./Scripts/build-local-dmg.sh
```

The resulting DMG is written under **dist/**. For this release the expected name is normally:

```
dist/Testudo-0.1.7-macOS-arm64.dmg
```

2.5 Updating

If the source checkout is clean:

```
cd ~/Testudo
git pull --ff-only
./Scripts/build-local-dmg.sh
```

Replace the previous copy in **/Applications** with the newly built application.

Build and install Testudo, then confirm that the application identifies itself as 0.1.7 build 8. If you maintain a development checkout, also inspect **Scripts/version.sh** and confirm that it contains the same version and build number.

First Launch and the Local Testudo Profile

3.1 Create or restore a profile

On first use, Testudo presents **Create or Restore Your Profile**. The local profile belongs to this installation, not to a Work Environment.

The setup interface can record professional information including:

- first, middle, last and preferred name;
- academic title and job title;
- professional email, phone and office;
- website, ORCID, LinkedIn and GitHub;
- professional address, city, postal code and country;
- professional fields, responsibilities and notes.

Only enter fields that are useful to you. The profile can later be edited from the application settings.

3.2 Importing a portable profile

If you previously exported a `.testudouser` file, you can import it instead of re-entering profile data.

A `.testudouser` file restores profile information and avatar data. It does *not* restore:

- the application password;
- registered Work Environment locations;
- Environment passwords;
- remembered Environment sign-ins.

3.3 Application password and recovery

After creating the profile, Testudo can protect the installation with an application password. The setup explicitly states that this password is separate from every Work Environment password.

The application password must contain at least eight characters. Recovery uses three user-defined security questions.

Security-question answers are stored as salted hashes. Matching normalises capitalization, accents and repeated spaces.

Create a disposable local profile, configure the application password and three recovery questions, use **Log Out of Testudo**, and log back in. Confirm that your registered Environment list is still present. Then compare this with **Sign Out from Testudo...**, which is intentionally much more destructive to local installation state.

Local Log Out, Local Sign Out and Environment Sign Out

Testudo exposes several actions that sound similar but have different scopes.

4.1 Log Out of Testudo

Logging out locks the application. Your local profile, registered Work Environments and remembered state remain available after the next login.

Use this when you simply want to protect the application while keeping the current installation configured.

4.2 Sign Out from Testudo

This is an installation-level reset for the local Testudo identity. The confirmation screen requires typing **SIGN OUT**.

It removes local information from this Mac, including the local profile, application password, registered Work Environments, local bookmarks, Environment identity mappings and remembered sign-ins.

It does **not** delete or modify the actual `.testudoenv` packages.

Before signing out, Testudo offers **Export My Profile Before Signing Out...**

4.3 Sign Out of Environment

This action affects the active Work Environment session rather than the local application account. It is useful when the same local Testudo installation is used to enter an Environment with different credentials.

If you only want to lock the application, use **Log Out of Testudo**. If you want to remove local Testudo-user state from the Mac, use **Sign Out from Testudo...**. If you only want to end the current Environment session, use **Sign Out of Environment**.

Work Environments

5.1 The Work Environments screen

After local authentication, Testudo shows **Work Environments** with the message *Choose where you want to work*. Registered Environments appear as cards showing their name and storage path.

Available actions are:

- **Create New Environment** ;
- **Open Existing Environment...** ;
- **Load Demo Environment** .

5.2 Creating an Environment

The creation sheet asks for:

- Environment name;
- initial Administrator username;
- Administrator password;
- password confirmation.

The first account belongs to the Work Environment and is separate from the local Testudo profile.

The Environment is saved as a portable `.testudoenv` package. You can choose a local, cloud-synchronised or external storage location.

Internally, a normal package contains files such as:

```
Example.testudoenv/  
|-- EnvironmentManifest.json  
|-- EnvironmentData.json  
|-- EnvironmentCredentials.json  
'-- EnvironmentIcon.png      # optional
```

5.3 Opening an existing Environment

Use **Open Existing Environment...** and select a `.testudoenv` package. Testudo validates the package and registers its location with the current installation.

If the Environment requires identity selection, Testudo presents the Environment login flow.

5.4 Environment login

The entry sheet contains a username, password and **Stay signed in** option.

When **Stay signed in** is enabled, this Mac can reopen the Environment without asking for those credentials until the user explicitly signs out of the Environment.

5.5 Forgetting an Environment from one Mac

In Environment settings, removing an Environment from Testudo only forgets its registration on that installation. The `.testudoenv` package and all Environment data remain at the storage location and can be reopened later.

Create a disposable Environment called *Manual Test*. Close it, return to Work Environments, and reopen it. Then remove its registration from Testudo and use **Open Existing Environment...** to register the same package again. Verify that the Work data is still present.

Environment Membership and Administration

6.1 Environment roles

Environment memberships currently have two roles:

- Administrator;
- User.

A membership also has an active/inactive state.

6.2 Membership identity

An Environment membership can either be linked to an existing Person entity or maintain its own first and last name. When linked to a Person, the membership name follows the Person.

The management view exposes:

- Linked Person;
- first and last name when unlinked;
- username;
- immutable Membership ID;
- Environment role;
- active membership state.

6.3 Environment passwords

Environment passwords are separate from the local application password.

An Administrator can assign a new Environment password without knowing the member's current password. Security-question answers are never revealed to Administrators.

6.4 Environment recovery questions

A member can configure three recovery questions for the current Environment account. These recovery questions belong to that Environment account and travel with the `.testudoenv` package.

If a member cannot answer a recovery question, an Environment Administrator can still assign a new password.

In a disposable Environment, create or inspect a second membership. Confirm that changing the Environment password does not change the Testudo application password. If the membership is linked to a Person, edit the Person name and verify that the membership display follows the linked Person.

The Three-Column Interface

7.1 Sidebar

The left sidebar is the global navigation layer. In 0.1.7 it contains:

- New Entry;
- Today;
- Calendar;
- All Tasks;
- To Do;
- In Progress;
- Completed;
- Discontinued;
- Archive;
- Timeline;
- Themes;
- Organizations;
- Groups;
- People.

7.2 Middle column

The middle column shows the collection selected in the sidebar: a Task list, Archive tree, calendar, Theme list, entity list or other contextual collection.

7.3 Detail pane

The right pane shows the selected object's detail view or a creation/editing form. Normal detail views are mostly read-only and expose a single whole-object **Edit** action where appropriate.

7.4 Sidebar visibility

Testudo uses the native macOS sidebar behaviour. The system-provided sidebar control hides and restores the first column.

7.5 Back and Forward navigation

The detail pane maintains browser-style navigation history through Back and Forward controls and two-finger trackpad navigation.

The navigation history represents the complete three-column context. When you move backward or forward, Testudo restores:

- the sidebar section;
- the middle-column selection;
- the right-pane detail content.

Archived Tasks participate in the same history as active Work.

Open an active Task, then open **Archive** and select a Closed Task, then open a Person. Use Back and Forward or the trackpad gesture. Verify that each step restores not only the right pane but also the corresponding sidebar section and middle-column selection.

Work: Tasks, Notes and Activities

8.1 Three Work types

Task Actionable work with a Task status and optional lifecycle dates.

Note Durable context that does not require Task status.

Activity A work-log item recording something that happened, optionally at a specific date and time.

Only Tasks can contain child Work. A Task may contain sub-tasks, Notes and Activities, and sub-tasks can contain further Work without a fixed nesting limit.

8.2 Creating Work

Choose **New Entry** or press Command-N. The Work editor includes sections for content, classification, parentage, timing and semantic relationships.

Common fields include:

- Type;
- one or more Themes;
- Parent Task;
- optional Title;
- Description or Note / What happened? text;
- relationships to People, Groups and Organizations.

At least a title or description is required.

8.3 Editing existing Work

Open the Work item and choose **Edit**. The same complete editor is used for Tasks, Notes and Activities. Existing values are pre-filled.

8.4 Creating child Work quickly

Inside a Task detail page, the **Contains** area can create:

- Sub-task;
- Note;
- Activity.

Activities created this way are still Work-log objects, not Calendar Events.

Create a top-level Task called *Prepare field campaign*. Add a sub-task called *Check instruments*, a Note with background information and an Activity recording a completed phone call. Then add a deeper sub-task under *Check instruments*. Navigate up and down the hierarchy and confirm that each item retains its own detail page.

Task Workflow Status

9.1 The four statuses

In Testudo 0.1.7, the user-facing Task statuses are exactly:

- To Do;
- In Progress;
- Completed;
- Discontinued.

Closed is deliberately absent from this list because it is an archive lifecycle action rather than a workflow outcome.

9.2 To Do

Use To Do for work that exists but has not begun. A Task can still have a deadline while remaining To Do.

9.3 In Progress

Use In Progress when work has begun. When a Task changes from To Do to In Progress and does not already have a Started date, Testudo can record the transition time automatically. A manually entered Started time remains independently editable.

9.4 Completed

Use Completed when the work ended successfully. The completion timestamp is preserved independently from Created and Started.

9.5 Changing status from the detail view

The status control allows normal workflow changes. Discontinued requires additional metadata, so it is intentionally handled through the full Work editor rather than a shortcut that could omit the reason.

Create a Task as To Do. Change it to In Progress and check its Started value. Then mark it Completed and confirm that a completion time appears. Reopen the editor and verify that Created, Started and Completed represent different concepts and can describe historical work accurately.

Discontinued Tasks

10.1 Why Discontinued exists

Not every Task finishes successfully. Work may fail, become unnecessary, be replaced by another approach, or prove infeasible. Deleting such a Task loses useful history; marking it Completed is misleading. Testudo therefore provides a terminal **Discontinued** status.

10.2 Required metadata

When the Task status is Discontinued, the editor requires a reason. Available reasons are:

- Failed;
- Superseded;
- No longer needed;
- Not feasible;
- Other.

The editor also includes:

- Discontinued date/time with source time zone;
- optional Outcome / note.

The discontinued time cannot be in the future and cannot be earlier than Started when a Started time exists.

10.3 What remains visible

The normal detail page shows the Discontinued time, reason and optional outcome. The Task also appears in the **Discontinued** sidebar section while it remains active rather than archived.

A discontinuation lifecycle event is retained in History.

10.4 Historical cycles

If a Task later returns to active work, earlier discontinuation history remains preserved. A later discontinuation can therefore represent a new lifecycle cycle without erasing the previous one.

Create a Task called *Use vendor API A*. Start it, edit it to Discontinued, choose *Superseded*, and write *Replaced by API B*. Confirm that the Task appears in **Discontinued**, that its detail page shows the reason and outcome, and that its Log contains a discontinuation event.

Closing, Archive and Reopening

11.1 Closed is not a status

Closing is an archival lifecycle operation. Only a **top-level Task** can be Closed, and it must already be terminal:

- Completed; or
- Discontinued.

A To Do or In Progress Task cannot be closed directly.

11.2 What closing does

Closing a top-level Task:

- records a Closed timestamp and source time zone;
- retains the real workflow status;
- moves the whole Task subtree to Archive;
- records a Closed lifecycle History event;
- removes the subtree from active Work queries.

Children do not independently become Closed. They are archived because their top-level ancestor is closed.

11.3 Archive presentation

The Archive middle column displays Closed top-level Tasks and their retained subtree. The root still indicates whether the actual outcome was Completed or Discontinued, and its context shows the Closed time.

11.4 Reopen

A Closed top-level Task exposes **Reopen**. Reopening removes the archive marker, returns the tree to active views, preserves the actual Completed or Discontinued status and records a Reopened History event.

11.5 Backward compatibility

Testudo 0.1.6 stored Closed as a Task status. Testudo 0.1.7 keeps a legacy persistence-only value so older Environment data can be decoded, then migrates the item to the separate archive lifecycle while preserving the most meaningful previous workflow state available.

Perform the following in a disposable Environment:

1. Create a top-level Task and a child Note.
2. Mark the Task Completed.
3. Use **Close** in the detail header.
4. Confirm that the Task disappears from active lists and appears in **Archive** with its child Note.
5. Open the archived Task and verify that its Status is still Completed.
6. Use Back/Forward navigation and confirm Archive selection is restored correctly.
7. Choose **Reopen**.
8. Confirm that the Task returns to active views as Completed and that Closed/Reopened events are preserved in its Log.

Repeat the exercise with a Discontinued Task and verify that its discontinuation reason and outcome survive Close and Reopen.

Notes

Notes are informational Work items. They do not have Task status and are useful for durable context that belongs near the work.

A Note can have:

- title and body;
- one or more Themes;
- a Parent Task;
- a Reminder;
- semantic relationships.

Use a Note when the item should remain available as context but does not represent work that progresses through To Do, In Progress, Completed or Discontinued.

Inside a Task, create a Note containing a decision or reference, enable a Reminder for tomorrow, and save it. Confirm that it appears under the parent Task and that the reminder is represented in Calendar and relevant Today views when its date becomes current.

Activities and the Work Log

13.1 Activity semantics

An Activity records something that happened while working. Examples include a discussion, measurement, intervention, review, decision or resolved problem.

The Activity editor provides an optional **Occurred** date/time with source time zone.

Activity vs Calendar Event

An Activity belongs to the Work history. A Calendar Event belongs to the Calendar domain. A meeting that is planned for tomorrow is a Calendar Event; after it occurs, you may also record an Activity containing the useful outcome.

13.2 Activity in Timeline and History

Activities contribute to chronological views and appear in the Work Log. A parent Task can therefore preserve a running record without rewriting its description repeatedly.

Create a Calendar Event for a meeting and link it to a Task. After the simulated meeting, create an Activity under the Task called *Meeting outcome* and record a historical Occurred time. Compare how the Calendar Event and Activity appear in their respective views.

Themes and Multi-Theme Classification

14.1 Purpose

Themes classify Work semantically and independently from the Task hierarchy and professional entity hierarchy. A Theme can have a parent Theme, allowing arbitrary Theme depth.

14.2 Theme detail fields

The current Theme interface exposes information including:

- Name and Parent Theme;
- Summary and Notes;
- Start and Target dates;
- Tags;
- URL;
- SF Symbol;
- related People, Groups and Organizations;
- Created and Updated timestamps.

The normal detail page is read-only and exposes a whole-object **Edit** action.

14.3 Multiple Themes on one Work item

A Work item can belong to several Themes simultaneously. This avoids duplicating the same Task simply because it belongs to multiple conceptual domains.

14.4 Descendant aggregation

Related Work for a parent Theme includes Work assigned to descendant Themes recursively. This aggregation does not rewrite direct membership.

Create Theme A, then child Theme B. Create a Task assigned directly to Theme B and also to a second independent Theme. Open Theme B and Theme A and verify that the Task appears in Related Work for both levels of the A/B hierarchy, while the Task editor still shows its actual direct Theme selections.

People

People are first-class professional entities with richer profile data than a simple name string.

15.1 Person profile fields

The Person editor can represent:

- first, middle, last and preferred name;
- academic title and position;
- primary and secondary email/phone;
- office and assistant/contact;
- staff or employee ID;
- address and workplace;
- website, LinkedIn and GitHub;
- ORCID, ResearcherID, Scopus Author ID and Google Scholar;
- fields/expertise, responsibilities and tags;
- preferred language and time zone;
- notes;
- Organization and Group affiliations.

15.2 Affiliations

A Person can belong to multiple Organizations and Groups. One affiliation can be primary. Some affiliations can be shown as automatic when implied by structure or membership data.

15.3 Environment membership link

A Person may be linked to an Environment membership. The Person detail page then exposes membership identity, role, active/inactive state and whether a local Environment password is configured.

A Person linked to an Environment membership cannot be deleted until the membership is unlinked.

Create a Person with a preferred name and ORCID, affiliate the Person with an Organization and a Group, then relate a Task to that Person using **With**. Open the Person and confirm that the Task appears in Related Work.

Organizations and Groups

16.1 Organizations

Organizations model institutional structure such as universities, institutes, departments, companies or observatories. Organizations can themselves be contained in other Organizations.

The editor can expose fields such as full name, short name, website, email, phone, address, city, postal code, country, tags, symbol, dates and notes.

16.2 Groups

Groups model teams, laboratories, committees, collaborations and working groups. A Group may be independent or contained by one or more compatible structural entities according to the current hierarchy rules.

16.3 Affiliated People

Organization and Group editors can manage affiliated People. Related Work can aggregate relevant work from descendant structures while suppressing duplicates.

Create an Organization called *Institute*, a child Organization called *Department*, and a Group called *Lab* inside the Department. Affiliate a Person with the Lab. Relate a Task to the Lab and verify Related Work from the relevant structural views.

Work Relationships

Work relationships attach professional context directly to Work items.

Available roles are:

- For;
- Requested by;
- With;
- Assigned to;
- Related to.

Each relationship points to a Person, Group or Organization.

17.1 Inheritance by child items

A relationship may be marked **Inherited by child items**. Child Work can then inherit relevant context from its parent Task.

When changing the Parent Task in the full editor, Testudo may ask whether to use the new Parent's inherited relationships. Copying them replaces the current related People, Groups and Organizations with the Parent relationships marked for inheritance.

Create a parent Task related **For** an Organization and enable inheritance. Create a child Task and inspect its inherited context. Then move the child to another Parent Task with a different inherited relationship and test the **Copy Parent Relationships / Keep Current Relationships** decision.

Related Work

People, Themes, Organizations and Groups share a Related Work presentation.

Related Work can contain Tasks, Notes and Activities. When related Work exists, the detail area can present the Work separately from the object's own inspector information.

Aggregation follows the relevant semantic structure:

- parent Themes can include descendant Theme Work;
- Organizations and Groups can aggregate from relevant descendants;
- duplicate Work is suppressed when several paths resolve to the same item.

Rows are navigable, so Related Work is also a bridge between the structural and Work domains.

Open a Person or Theme that has several related Work items. Select a related Task, then use Back. Verify that the original Person/Theme and its corresponding left/middle-column context are restored.

Today

19.1 Today is a work summary, not only a schedule

The Today view combines several kinds of current-day relevance. In 0.1.7 the interface includes sections such as:

- Needs Attention;
- In Progress;
- Upcoming;
- Done Today;
- Discontinued Today;
- Today's Log.

The underlying Today collection considers active Work whose Created, Logged/Occurred, Started, Completed or Discontinued date is today.

19.2 Needs Attention

Needs Attention highlights urgent active items such as overdue deadlines, work due today and relevant reminders. Closed Task trees are excluded because archived Work is not part of the active workflow.

19.3 Upcoming

Upcoming provides near-future actionable context, such as deadlines and reminders.

Create one Task due today, start another Task today, complete a third Task today, discontinue a fourth Task today and create an Activity with today's Occurred time. Open **Today** and identify where each item appears. Then Close one terminal top-level Task and confirm the archived tree no longer contributes to the active Today workflow.

Task Lists and Timeline

The sidebar exposes dedicated Task filters:

- All Tasks;
- To Do;
- In Progress;
- Completed;
- Discontinued.

These lists operate on active Work. Archived Task trees are excluded until reopened.

20.1 Timeline

Timeline focuses on chronological Work activity rather than a single workflow state. Activities and lifecycle dates help make long-running work understandable after the fact.

Create four Tasks, one in each workflow status. Confirm that each appears in its corresponding sidebar filter. Close the Completed and Discontinued top-level Tasks and verify that they move to Archive and disappear from the active status filters. Reopen them and verify that they return to their retained status filters.

History and the Work Log

21.1 Audit timestamps

The History section exposes key timestamps such as Created and Updated, plus type-specific lifecycle values such as Started, Completed, Discontinued or Occurred.

21.2 Log events

The chronological Log can represent:

- created;
- edited;
- status changed;
- scheduled;
- started;
- completed;
- discontinued;
- closed;
- reopened;
- activity logged;
- relationship added or removed;
- moved.

The Log is deliberately not only a mirror of the current state. Earlier lifecycle cycles remain meaningful historical records.

21.3 Deletion and history

Deleting a Work item is different from discontinuing or closing it. Deletion permanently removes the item and its descendants, plus associated relationships and history for those deleted Work items.

If a Task existed and its outcome matters, prefer Completed, Discontinued and possibly Close. Use deletion for data that should genuinely be removed rather than preserved as history.

Calendar Overview

22.1 What appears in Calendar

The Calendar combines several scheduling concepts:

- Calendar Events;
- Task deadlines;
- Note reminders.

The month view includes navigation for previous/next month and **Today** . Day summaries distinguish Events, Deadlines and Reminders.

Toolbar actions include:

- Connect iCal Calendar;
- Connect Apple / iCloud Calendar;
- Sync connected external calendars;
- New Calendar Event.

22.2 Calendar sources

Current source kinds include:

- Testudo local calendar;
- iCal Subscription;
- Apple Calendar / iCloud.

For the same date, create a Task deadline, a Note reminder and a Calendar Event. Open that date in Calendar and confirm that all three appear but are identified by their different scheduling roles.

Creating and Editing Calendar Events

23.1 New Calendar Event

A local Calendar Event can contain:

- Title;
- Calendar;
- All day flag;
- Start and End;
- source time zones for timed events;
- Location;
- Notes;
- Related Tasks;
- Related Themes.

The End cannot be earlier than Start.

Related Tasks are selected from the Task hierarchy, and related Themes are selected from the Theme hierarchy.

23.2 Calendar Event status

Calendar event statuses include Confirmed, Tentative and Cancelled where applicable to the event model.

23.3 External events

Events imported from external calendars are read-only for their external details. Testudo relationships can still be edited. The detail view explicitly states when an event is a read-only external calendar event.

Create a local Event called *Project review*, link it to a Task and a Theme, then open the Event and navigate through those links. Return with Back and verify the Calendar context is restored.

Apple Calendar Integration

24.1 Connecting Apple Calendar

Choose **Connect Apple / iCloud Calendar**. Testudo requests macOS Calendar access and lists calendars available through Apple Calendar on the Mac.

The connection view states that imported events are read-only in Testudo and that only Testudo relationships can be edited.

The initial synchronisation window covers approximately one year in the past through two years in the future.

24.2 Permission and availability

If macOS permission is not granted, or no compatible calendars are available, the connection view reports the condition. Permission is controlled by macOS privacy settings.

Connect a non-critical Apple Calendar, sync it and open one imported event. Confirm that event details are read-only while Related Work / Theme links remain manageable inside Testudo.

iCal Subscriptions

Choose **Connect iCal Calendar** and provide the secret address in iCal format. An optional Calendar name can be entered.

Events imported from an iCal subscription are read-only.

The secret subscription address is treated as sensitive and is stored in macOS Keychain rather than in ordinary Testudo data.

The Keychain credential is local to the Mac on which it was entered. A shared `.testudoenv` package stores only the identifier used to locate that local credential; it does not contain the Secret iCal address itself.

When the same Work Environment is opened on another Mac, the Calendar and its imported events may therefore already be present while the local Secret address is absent. Use **Restore Secret Address** to provide the credential on that Mac. Restoration keeps the existing Calendar identity and updates the existing imported subscription rather than creating a second Calendar.

A missing local Keychain credential is treated as a machine-local condition and is not written into the shared Work Environment as a synchronization error.

Anyone with a secret subscription URL may be able to read the calendar feed. Do not place the URL in screenshots, Notes or shared documentation.

If you have a disposable iCal feed, connect it, trigger external-calendar synchronisation and inspect an event. Verify that the source behaves as read-only and that the feed URL is not shown as ordinary Work Environment data.

Time Zones and Historical Dates

26.1 Absolute time and source zone

Testudo stores absolute timestamps as dates and, where relevant, stores a separate IANA time-zone identifier describing the zone in which the value was entered or recorded.

Time-zone-aware fields include, depending on type:

- Created and Updated;
- Scheduled;
- Deadline;
- Reminder;
- Started;
- Completed;
- Discontinued;
- Closed;
- Activity Occurred;
- Calendar Event Start and End.

The current device time zone is the normal display context, while the source zone helps preserve meaning across travel and historical entry.

26.2 Historical entry

Started, Discontinued and Activity occurrence times can describe events before the item was entered into Testudo, subject to logical validation rules.

Create an Activity with a historical time and explicitly choose another IANA time zone. Save it, then reopen the detail and editor. Verify that the absolute moment and source-zone context remain consistent.

Demo Environment

27.1 Purpose

Load Demo Environment creates a complete fictional data set intended for exploration. It includes People, Organizations, Groups, Themes, Tasks, sub-tasks, Notes, Activities, Calendar Events, affiliations, relationships, chronology, history and archived work.

Dates are generated relative to the date on which the Demo is created, so the environment remains useful over time.

27.2 Demo naming

Multiple Demo Environments can coexist. The generator uses names such as:

```
Demo
Demo-2
Demo-3
```

Available numbering can be reused after a Demo is removed.

27.3 Suggested tour

Use the Demo to inspect, in order:

1. Today;
2. one multi-level Task tree;
3. a Person with affiliations and Related Work;
4. a Theme with descendant-related Work;
5. Calendar;
6. Archive;
7. browser-style Back/Forward navigation across different object types.

Before upgrading an important real Environment, create a fresh Demo and perform the tour above. It is a low-risk way to detect obvious UI or persistence regressions.

Storage, Portability and Backups

28.1 Portable Environment package

The `.testudoenv` package is the principal portable unit. It contains the Environment's manifest, data, credentials and optional icon.

28.2 Cloud-synchronised folders

An Environment can live in Dropbox, iCloud Drive, OneDrive or another filesystem provider visible to macOS. Testudo still works on the package as a file package.

Avoid simultaneous conflicting edits from several Macs. Wait for file synchronisation to complete before opening the same package elsewhere.

28.3 Backups

File synchronisation is not a substitute for independent backup. A deletion or corruption can itself be synchronised.

Recommended practice:

1. keep at least one independent backup of important Environments;
2. before upgrading between early versions, copy critical `.testudoenv` packages;
3. avoid manually editing JSON inside production Environment packages;
4. if using cloud storage, confirm synchronisation has completed before moving computers.

Security and Privacy

29.1 Passwords

Application and Environment passwords are separate. Password verification uses salted password hashes rather than readable plaintext storage.

29.2 Security questions

Recovery answers are stored only as salted hashes. Administrators cannot view other users' recovery answers.

29.3 External calendar secrets

Secret iCal URLs use macOS Keychain integration rather than ordinary Environment data. The Secret value is machine-local; it is not transferred merely because a `.testudoenv` package is synchronized to another Mac. Since Testudo 0.1.9, the credential for an existing subscription can be restored without changing that Calendar's identity.

29.4 Sensitive portable files

A `.testudoenv` package may contain substantial professional context, including entities, relationships and history. Treat it as potentially sensitive when copying, sharing or backing it up.

Deletion Semantics

Deletion is permanent and differs from lifecycle transitions.

Task / Note / Activity Deleting a Work item removes it and descendant Work, together with associated relationships and history for the deleted items.

Theme Deleting a Theme can also delete descendant Themes and Work assigned inside that Theme hierarchy according to the confirmation shown by the application.

Organization / Group Deleting the entity removes structural memberships and Work relationships; contained entities themselves are retained where the application confirmation states so.

Person Deletion removes the Person profile, structural memberships and Work relationships, but is blocked while the Person is linked to an Environment membership.

Calendar Event Deletion permanently removes the event from Testudo; an externally synchronised deletion can be queued for synchronisation where supported.

Deletion scope differs by object type. Testudo's confirmation message is the authoritative description of what will be removed. For Tasks with meaningful history, prefer Completed, Discontinued and Archive instead of deletion.

Recommended Working Practices

31.1 Separate plan, context and evidence

A useful Testudo pattern is:

- Task for what must be done;
- Note for stable context or reference material;
- Activity for what actually happened;
- Calendar Event for reserved or scheduled time.

31.2 Use Discontinued instead of pretending success

If work ended without successful completion, use Discontinued with a reason and optional outcome. This gives later readers a truthful explanation.

31.3 Close only when the top-level work is no longer part of active workflow

A Completed or Discontinued Task can remain visible in active status lists for as long as it is still operationally relevant. Close it when the whole top-level Task tree belongs in Archive.

31.4 Use Themes for concepts, not as a duplicate Task tree

Themes are semantic classification. Parent Task relationships should represent Work decomposition, while Theme hierarchy should represent conceptual grouping.

31.5 Model recurring professional context explicitly

If the same Person, lab, institution or committee repeatedly appears, create an entity and relationship rather than repeatedly typing names into free text.

31.6 Record outcomes as Activities

For long-running work, short Activities documenting important events provide a more useful record than continuously rewriting the Task description.

Troubleshooting

32.1 The installer says the source checkout has local changes

The automatic installer deliberately refuses to overwrite local work. Inspect the checkout:

```
cd ~/Testudo
git status
```

Commit, stash or intentionally discard the local changes before using the automatic updater.

32.2 Apple developer tools are missing

Request Apple's Command Line Tools:

```
xcode-select --install
```

Then rerun the compatibility check or installer.

32.3 An Environment package was moved

If the registered path or security-scoped bookmark no longer points to the package, use **Open Existing Environment...** and select the package at its new location.

32.4 Environment login fails

Check:

- the correct Environment is active;
- the username is correct;
- the membership is Active;
- a password is actually configured for the membership;
- password recovery questions are configured if using recovery.

If recovery is unavailable, contact an Environment Administrator.

32.5 Apple Calendar does not appear

Check macOS Calendar permission, verify that the desired calendar is available in Apple Calendar, reconnect if necessary and run external-calendar synchronisation.

32.6 iCal synchronisation fails

Possible causes include an invalid or expired secret address, connectivity problems, changes at the calendar provider, or a Secret address that has not yet been stored in the current Mac's Keychain. If Testudo reports that this Mac does not have the Secret iCal address, use **Restore Secret Address** and enter the existing subscription URL. This restores the credential in place and should be preferred to disconnecting and recreating the Calendar when the existing Calendar identity and relationships must be preserved. If macOS asks for Keychain access after Testudo has been rebuilt, authorize the application and retry synchronization.

32.7 A Task cannot be Closed

Closing is available only for top-level Tasks whose status is Completed or Discontinued. Complete or Discontinue the Task first. Child Tasks are archived through their closed top-level ancestor.

32.8 A Discontinued status change appears unavailable in a short-cut

Discontinued requires a reason and date. Use the full Work editor so the required metadata can be entered and validated.

32.9 Archived Work is missing from active lists

This is expected. A Closed top-level Task and its subtree are removed from active Task, Today and related active-work queries. Open Archive or Reopen the root Task.

Guided End-to-End Verification Lab

This chapter provides a complete hands-on test that exercises the most important 0.1.7 behaviours in one disposable Environment.

33.1 Step 1: create the Environment

Create a new Environment named *Manual Verification*. Create its Administrator account and enter the Environment.

Expected result: a fresh Environment opens with no Work data other than any application defaults.

33.2 Step 2: create professional context

Create:

- Organization: *Example Institute*;
- Group: *Field Team*;
- Person: *Alex Example*;
- Theme: *Campaign 2027*.

Affiliate the Person with the Group and/or Organization as appropriate.

Expected result: each object has a read-only detail inspector and whole-object Edit action, and navigation links work between structural objects.

33.3 Step 3: create a top-level Task

Create Task *Prepare campaign* assigned to Theme *Campaign 2027*. Add a relationship **With** Alex Example and enable inheritance for child items. Set a future Deadline.

Expected result: the Task appears in To Do and in Related Work for the selected Theme/Person where appropriate.

33.4 Step 4: create children

Inside the Task create:

- sub-task *Check instruments*;
- Note *Shipping instructions*;
- Activity *Kick-off discussion* with an Occurred time.

Expected result: the Task's Contains section shows all three children and inherited relationships behave according to the relationship settings.

33.5 Step 5: exercise status and Today

Change *Check instruments* to In Progress. Confirm Started is populated or set manually. Create another Task due today.

Expected result: Today shows relevant active work in the appropriate Needs Attention / In Progress / Upcoming areas.

33.6 Step 6: exercise Discontinued

Create Task *Use legacy sensor*. Edit it to Discontinued, reason *Superseded*, outcome *New sensor selected*.

Expected result: the Task appears in Discontinued, its detail page shows the metadata, and the Log contains the lifecycle event.

33.7 Step 7: exercise Completed and Archive

Complete the top-level *Prepare campaign* Task, then choose Close.

Expected result:

- the complete subtree moves to Archive;
- the root still reports Status = Completed;
- active Task lists no longer include the tree;
- Close is present in history.

33.8 Step 8: navigation history

While viewing the archived Task, open a Person, then a Theme. Use Back twice and Forward twice.

Expected result: sidebar section, middle-column selection and right-pane content travel together through the history.

33.9 Step 9: Reopen

Return to the archived root and choose Reopen.

Expected result: the Task tree leaves Archive, returns to active views with Status = Completed, and the history contains both Closed and Reopened.

33.10 Step 10: calendar

Create a local Calendar Event, link it to the Task and Theme, and set a location. Also create a Note reminder and Task deadline on the same date.

Expected result: Calendar distinguishes Event, Reminder and Deadline while still allowing navigation to the related Work.

If all ten steps behave as described, you have exercised the core 0.1.7 data model, Task lifecycle, Archive semantics, three-column navigation, semantic relationships, Today, Calendar and persistence-facing workflows.

Technical Reference

34.1 Important file extensions

Extension	Purpose
<code>.testudoenv</code>	Portable Work Environment package.
<code>.testudouser</code>	Portable local Testudo profile information; does not contain Environment registry or passwords.
<code>.dmg</code>	Locally generated macOS installer disk image.

34.2 Source tree

```
Sources/Testudo/
|-- Auth/
|-- Demo/
|-- Models/
|-- Resources/
|-- Store/
|-- Time/
|-- Views/
'-- TestudoApp.swift
```

34.3 Build scripts

```
Scripts/build-app.sh
Scripts/build-release.sh
Scripts/build-local-dmg.sh
Scripts/build-manual.sh
Scripts/version.sh
```

34.4 Version identity

`Scripts/version.sh` is the build scripts' central version source. For this manual:

```
TESTUDO_VERSION="0.1.10"
TESTUDO_BUILD_NUMBER="11"
```

34.5 Task status persistence compatibility

The current user-visible Task status set excludes Closed. A legacy persistence-only Closed value remains decodable so Environment files written by Testudo 0.1.6 and earlier can be migrated to the separate `closedAt` archive lifecycle.

Glossary

Activity A Work item recording something that happened, optionally with an Occurred timestamp.

Archive The view containing top-level Tasks that have been Closed, together with their retained subtrees.

Calendar Event A scheduled calendar-domain object, distinct from Activity.

Closed An archive lifecycle state for terminal top-level Tasks; not a user-facing Task workflow status.

Completed A terminal Task status representing successful completion.

Discontinued A terminal Task status representing work that ended without successful completion and carries a reason/date.

Entity A Person, Group or Organization.

Environment Administrator A membership with administrative privileges inside one Work Environment.

Environment membership An identity belonging to one Work Environment, separate from the local Testudo profile.

Group A laboratory, team, committee, collaboration or similar collective entity.

Local Testudo profile The installation-level professional profile of the Testudo user.

Note An informational Work item without Task status.

Organization An institutional entity that can participate in structural hierarchy.

Person An individual represented within a Work Environment.

Related Work Work associated with a Theme or professional entity directly or through supported descendant aggregation.

Reopened A lifecycle event that returns a Closed top-level Task from Archive while preserving its real workflow status.

Task An actionable Work item with one of the user-facing statuses To Do, In Progress, Completed or Discontinued.

Theme A hierarchical semantic classification independent of Work and entity hierarchies.

Work Environment An independent portable workspace represented by a `.testudoenv` package.

Work relationship A semantic link between Work and a Person, Group or Organization.

Command Reference

A.1 Check compatibility

```
curl -fsSL \  
https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh \  
| bash -s -- --check
```

A.2 Install

```
curl -fsSL \  
https://raw.githubusercontent.com/foivoskar/Testudo/main/install.sh \  
| bash
```

A.3 Clone manually

```
git clone https://github.com/foivoskar/Testudo.git  
cd Testudo
```

A.4 Update source

```
git pull --ff-only
```

A.5 Build local installer

```
./Scripts/build-local-dmg.sh
```

A.6 Build application only

```
./Scripts/build-app.sh
```

A.7 Request Apple Command Line Tools

```
xcode-select --install
```

Manual Source and Compilation

The manual source lives under:

```
Documentation/TestudoManual/
```

The maintained source files are:

```
Testudo_manual.tex  
testudomanual.cls
```

This edition deliberately keeps `testudomanual.cls` unchanged.

The normal repository-level build command is:

```
./Scripts/build-manual.sh
```

The script compiles the manual with XeLaTeX using the existing class file and writes the finished PDF at the repository root. A direct build remains possible:

```
cd Documentation/TestudoManual  
latexmk -xelatex Testudo_manual.tex
```

The repository-level final copy is named:

```
Testudo_manual.pdf
```

Quick Acceptance Checklist

Use this checklist after installing a new Testudo build:

1. Local login works and Settings are accessible.
2. A Work Environment can be opened or created.
3. New Entry creates Task, Note and Activity correctly.
4. Task status filters show To Do, In Progress, Completed and Discontinued.
5. Discontinued requires a reason and preserves outcome metadata.
6. Only terminal top-level Tasks expose Close.
7. Archive preserves the actual Completed/Discontinued status.
8. Reopen returns the subtree to active views.
9. Back/Forward restores all three columns, including Archive destinations.
10. Today reflects current-day Work lifecycle activity.
11. Calendar shows Events, deadlines and reminders.
12. External calendar events remain read-only except for Testudo relationships.
13. Person/Theme/Organization/Group Related Work navigation works.
14. A disposable Environment can be closed and reopened without data loss.

Early Release Notice

Testudo is under active development. Screens, labels and individual workflows may evolve between early versions. Before upgrading important data, keep an independent copy of critical `.testudoenv` packages.

License

Testudo is distributed under the MIT License. See the repository's [LICENSE](#) file for the complete text.

Author

Foivos Karakostas



Testudo User Manual
Manual version 1.6
October 2026